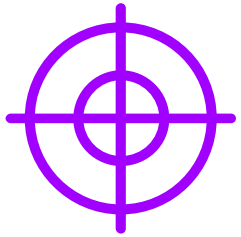
# WebDriver Architecture

# Module Objectives

At the end of this course, you will be able to:



- Understand about WebDriver Architecture
- Demonstrate how to use Maven Project with Dependencies
- Demonstrate Web Automation with WebDriver and TestNG
- Demonstrate how to design Browser Utility Class

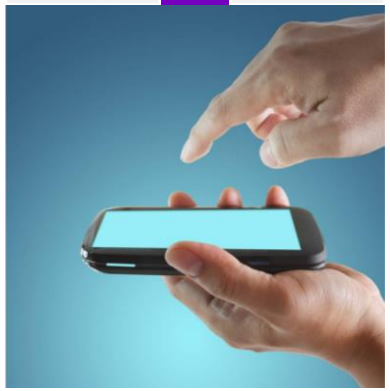
Copyright © 2024 Accenture. All rights reserved.

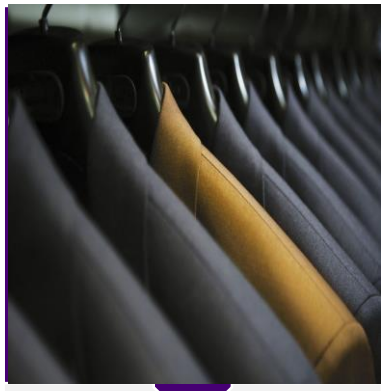
# Ground Rules

For a successful class, please:

Arrive on time

Keep your cell phones off





Wear business formals

Assist and show respect to your colleagues irrespective of skill and knowledge level


Do not use the web for personal reasons during training sessions

Adhere to attendance policy as directed by training coordinator


Copyright © 2024 Accenture. All rights reserved.

# WebDriver Architecture

01 Introduction to WebDriver Architecture

02 Maven Project with Dependencies

03 Web Automation with WebDriver and TestNG

04 Browser Utility Class

Copyright © 2024 Accenture. All rights reserved.

# Key New Features Introduced in Selenium 4

1. **Selenium WebDriver is now W3C Compliant**
   WebDriver follows W3C standards, improving cross-browser compatibility and stability.
   JSON Wire Protocol is no longer needed, simplifying communication with browsers.
2. **Relative Locators**
   Allows finding elements based on visual relationships (e.g., above, below, near).
   Helps locate elements dynamically without relying on complex XPath or CSS Selectors.
3. **Better Window and Tab Management**
   Selenium 4 enables creating and switching between windows/tabs in the same session.
   No need to open a new WebDriver instance, improving test efficiency.
4. **Upgraded Selenium Grid**
   New Grid UI and support for Docker-based deployments enhance test execution.
   Provides better observability, improved logging, and distributed execution.

Copyright © 2024 Accenture. All rights reserved.

# Key New Features Introduced in Selenium 4

1. **Upgraded Selenium IDE**
   Selenium IDE now supports improved debugging, better locators, and parallel execution.
   It comes as a browser extension with better export options for scripts.
2. **No Need for System.setProperty()**
   Selenium 4 eliminates the need for manually setting driver paths.
   WebDriverManager handles browser driver management automatically.
3. **New Features in the Actions Class**
   Supports multiple key inputs and precise touch interactions for better automation.
   New methods like clickAndHold(), contextClick(), and doubleClick() are improved.

Copyright © 2024 Accenture. All rights reserved.

# Introduction to WebDriver Architecture

# Introduction to WebDriver Architecture (1)

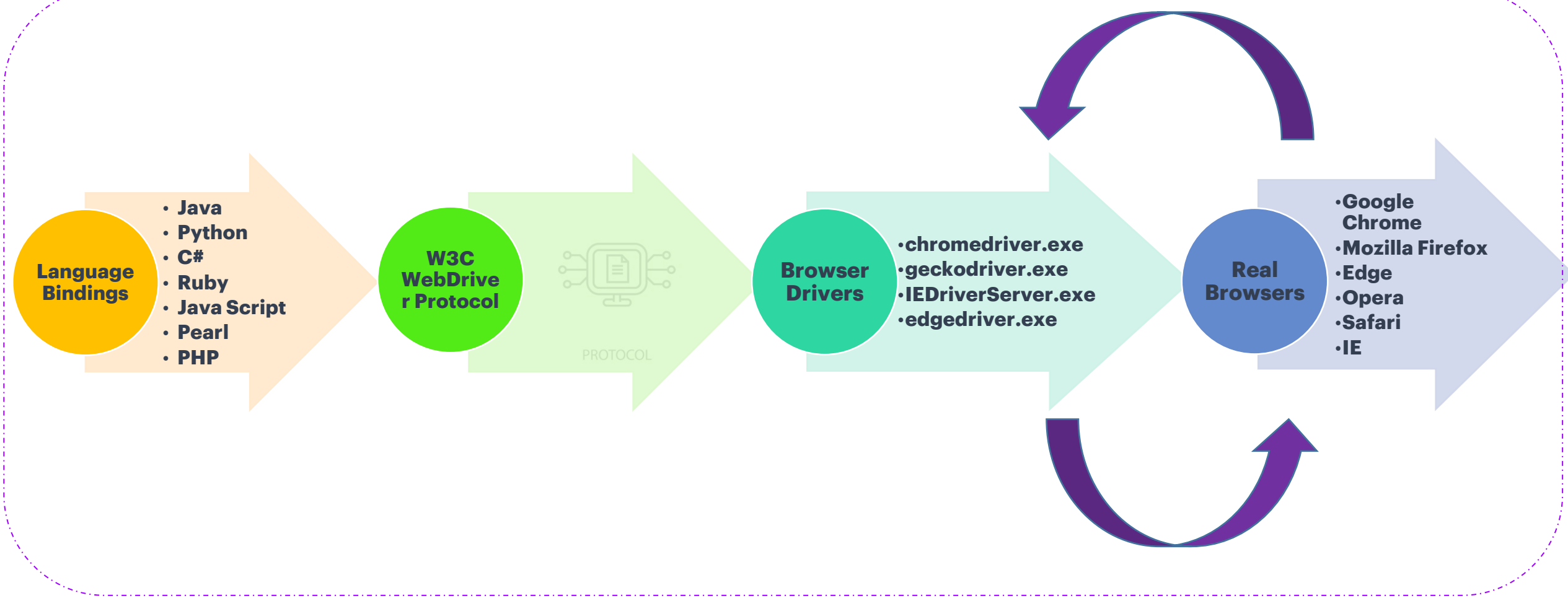
## Selenium WebDriver Architecture

- WebDriver API is the latest and enhanced library to interact with web applications
- Selenium scripts using WebDriver API can be implemented in different programming languages like Java, C#, Python, Ruby, JavaScript, Pearl, and PHP
- Features of WebDriver API:
  - Object-oriented way of using the library
  - Faster in communication from test scripts to real browsers because of native browser support
  - Better support for dynamic web pages & AJAX-based applications
  - Enhanced Library for supporting keyboard and mouse actions

Copyright © 2024 Accenture. All rights reserved.

# Introduction to WebDriver Architecture (2)

## Selenium WebDriver Architecture




Copyright © 2024 Accenture. All rights reserved.

# Maven Project with Dependencies

# Maven Project with Dependencies (1)

## Overview of Maven Project

- Apache Maven is a project management tool, which can be used for writing, managing, and execution of development and test code
- It can manage the project's build, documentation, and report in a central place
- It provides a complete build lifecycle framework for developers and test automation engineers
- The main difference between Maven Project and Java Project:
  - **Java project** requires to download JAR files separately and configure in the project. Any update in the version must be downloaded again and configured. This involves a lot of rework
  - **Maven project** builds and automatically downloads required JAR files, based on dependencies specified in the POM.xml file. But maven project requires an open internet connection to download the required JAR files for the first time. It downloads and stores the JAR files in the local .m2 repository

Copyright © 2024 Accenture. All rights reserved.

# Maven Project with Dependencies (2)

## Overview of Maven Project

- Maven repository is a directory, which stores all the project's JARs, library JARs, and plugins
- These JARs can be easily accessed and updated by Maven only
- Maven repository is of 3 types:
  - Local
  - Central
  - Remote

Copyright © 2024 Accenture. All rights reserved.

# Maven Project with Dependencies (3)

## Overview of Maven Project

- Maven Local Repository (.m2 folder):
  - It is a folder location in the machine
  - By default, it is created by Maven in %USER_HOME% directory
  - On Maven build, it creates the local repository and automatically downloads all the dependency jars and plugins into the local repository

- Maven Central Repository (https://mvnrepository.com/artifact):
  - It is a repository that contains many commonly used libraries. It is maintained by the Maven community
  - When Maven does not find any dependency in the local repository, it searches and downloads from the central repository

Copyright © 2024 Accenture. All rights reserved.

# Maven Project with Dependencies (4)

## Overview of Maven Project

- Maven Remote Repository:
  - It is the developer's custom repository to store the dependencies and plugins
  - Maven stops the build process with an error when it cannot find a dependency in Central Repository
  - In such situations, we can force maven to download the dependency from Remote Repository

Copyright © 2024 Accenture. All rights reserved.

# Maven Project with Dependencies (5)

## Overview of pom.xml

- **pom.xml** is the core of a maven project's configuration. It contains a lot of information that is required to build a maven project
- **groupId** → Id of the project's group. Generally, a package name
- **artifactId** → Id of the project. Generally, a project name
- **plugins**
  - maven-compiler-plugin → Used to compile the source code of the project
  - maven-surefire-plugin → Forces the pom.xml to execute the Cucumber runner class, which in turns executes the Test Classes
- **dependencies**
  - allows the maven to download and use the required artifacts/jars from the local or central repository
- Execute a maven project using pom.xml
  - pom.xml can be run by specifying the maven goals
  - **clean** → Used to delete the build directory (build output or test results)
  - **test** → Used to test the source code with help of testing framework (JUnit, TestNG)

Copyright © 2024 Accenture. All rights reserved.

# Maven Project with Dependencies (6)

## pom.xml components

```
    <dependency>
        <groupId>org.seleniumhq.selenium</groupId>
        <artifactId>selenium-java</artifactId>
        <version>4.22.0</version>
    </dependency>
    <!-- https://mvnrepository.com/artifact/org.testng/testng -->
    <dependency>
        <groupId>org.testng</groupId>
        <artifactId>testng</artifactId>
        <version>7.9.0</version>
        <scope>test</scope>
    </dependency>
</dependencies>
```

**<dependencies> are used to configure java libraries like selenium, TestNG etc..**

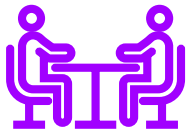
**<build> is used to configure plugins like java compiler, surefire etc..**

```
<build>
    <plugins>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-compiler-plugin</artifactId>
            <version>3.10.1</version>
        </plugin>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-surefire-plugin</artifactId>
            <version>3.3.0</version>

        </plugin>
    </plugins>
</build>
```

Copyright © 2024 Accenture. All rights reserved.

# Maven Project with Dependencies (7)

## Demo: Create a simple maven project with dependencies and plugins



- Priya has learned about the Maven project and the importance of the pom.xml file
- She decides to create a Maven project and set up all the necessary artifacts in the pom.xml file
- Scenario

**Step 1** Create a simple Maven project by providing Group Id and Artifact Id

**Step 2** Configure pom.xml file with “maven-compiler” and “maven-surefire” plugins

**Step 3** Configure pom.xml file with “Selenium JAVA” and “TestNG” dependencies

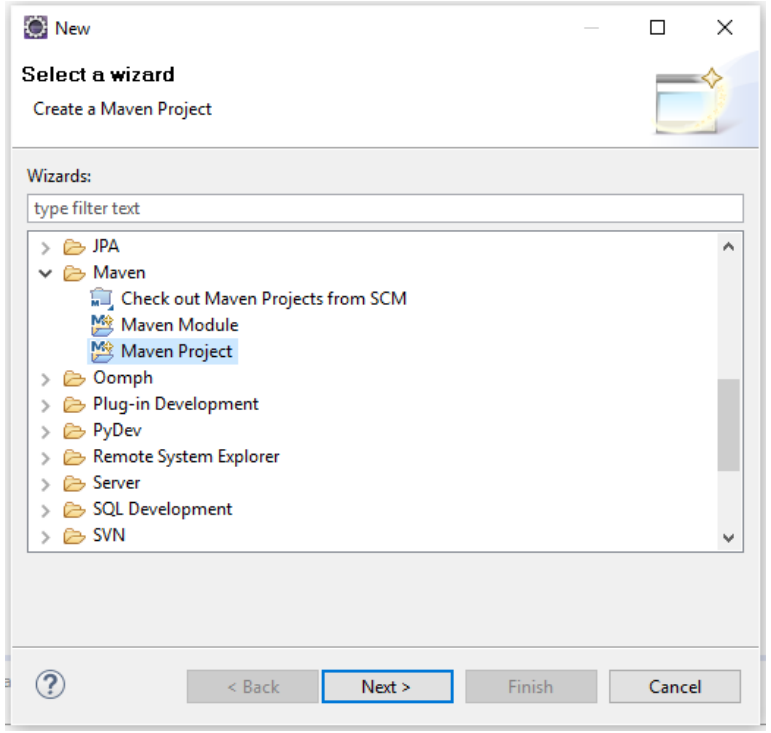
**Step 4** Save and Update the project

Copyright © 2024 Accenture. All rights reserved.

# Maven Project with Dependencies (8)

Demo Solution: Create a simple maven project with dependencies and plugins (1)

**Step 01:** Go to **File → New → Project**
**Step 02:** Search for "**Maven Project**"

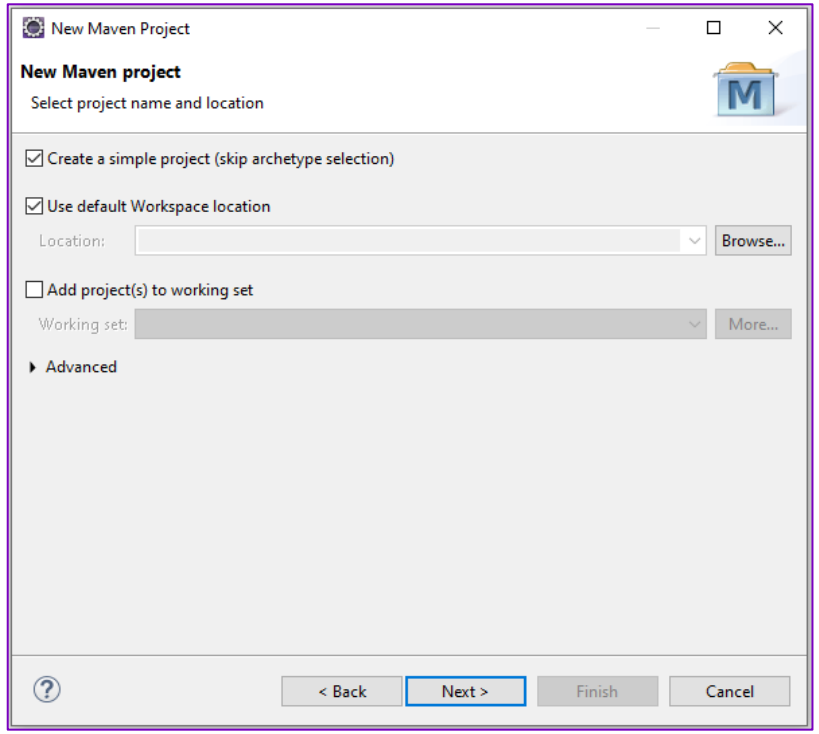



Copyright © 2024 Accenture. All rights reserved.

# Maven Project with Dependencies (9)

## Demo Solution: Create a simple maven project with dependencies and plugins (1)

**Step 03:** Click **Next** and the **New Maven Project** window opens
**Step 04:** Select **Create a simple project** and Click **Next**

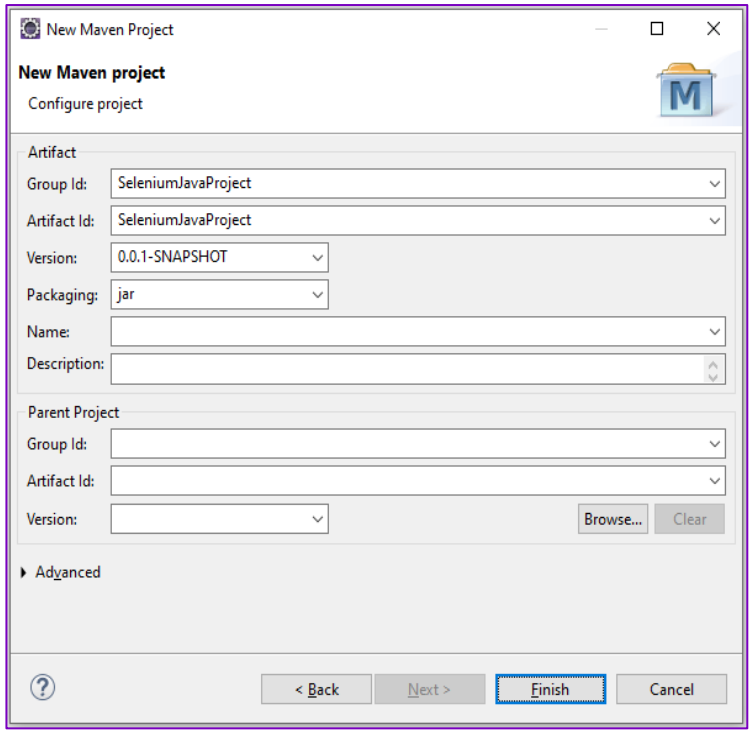



Copyright © 2024 Accenture. All rights reserved.

# Maven Project with Dependencies (10)

Demo Solution: Create a simple maven project with dependencies and plugins (1)

**Step 05:** Enter **Group Id** (package name) and **Artifact Id** (project name)
**Step 06:** Click **Finish**




Copyright © 2024 Accenture. All rights reserved.

# Maven Project with Dependencies (11)

Demo Solution: Create a simple maven project with dependencies and plugins (1)

**Step 07:** Open the pom.xml file and configure the following:

Maven-compiler plugin
Maven-surefire plugin
Selenium JAVA dependency
TestNG dependency


**Step 08:** Save the Maven project (CTRL + S)
**Step 09:** Update the Maven project (ALT + F5) to remove any project errors

Copyright © 2024 Accenture. All rights reserved.

# Web Automation with WebDriver and TestNG

# Web Automation with WebDriver and TestNG (1)

## Browser Control - Instantiating browsers using drivers

```
WebDriver driver= new ChromeDriver();
```

Code to launch Chrome Browser

```
WebDriver driver= new EdgeDriver();
```

Code to launch Edge Browser

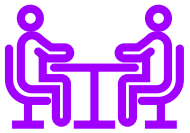
```
WebDriver driver= new FirefoxDriver();
```

Code to launch Firefox Browser

Copyright © 2024 Accenture. All rights reserved.

# Web Automation with WebDriver and TestNG (2)

## Demo: Executing Selenium Test Scripts in Chrome Browser



- Priya has learned the Selenium WebDriver commands to automate the Chrome browser
- She decides to create a new Maven project and design a script to automate the browser
- Scenario:

**Step 1** Create a TestNG class to write the test automation webdriver commands

**Step 2** Pre-requisites → Launch Chrome browser and maximize the browser window

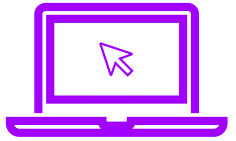
**Step 3** Test Method → Open AccenSportz
URL: **https://lkmdemoaut.accenture.com/AccenSportz/#/**

**Step 4** Post-conditions → Close the Chrome browser

Copyright © 2024 Accenture. All rights reserved.

# Web Automation with WebDriver and TestNG (3)

## Demo Solution: Executing Selenium Test Scripts in Chrome Browser

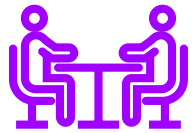


```
1  package demo1;
2
3  import org.openqa.selenium.WebDriver;
4  import org.openqa.selenium.chrome.ChromeDriver;
5  import org.testng.annotations.AfterClass;
6  import org.testng.annotations.BeforeClass;
7  import org.testng.annotations.Test;
8
   Run All
9  public class Browserlaunch {
10     WebDriver driver;
11
12     @BeforeClass
13     public void beforeclass() {
14         // Initiating browser
15         driver = new ChromeDriver();
16         // maximize the browser
17         driver.manage().window().maximize();
18     }
19
20     @Test
       Run | Debug
21     public void test01() {
22         // Navigating to the application
23         driver.get("https://lkmdemoaut.accenture.com/AccenSportz/#/");
24     }
25
26     @AfterClass
27     public void afterclass() {
28         // closing the browser
29         driver.close();
30     }
31
32 }
```

Copyright © 2024 Accenture. All rights reserved.

# Web Automation with WebDriver and TestNG (4)

## Demo: Executing Selenium Test Scripts in Edge Browser



- Priya has learned the Selenium WebDriver commands to automate the Edge browser
- She decides to create a new Maven project and design a script to automate the browser
- Scenario:

**Step 1** Create a TestNG class to write the test automation webdriver commands

**Step 2** Pre-requisites → Launch Edge browser and maximize the browser window

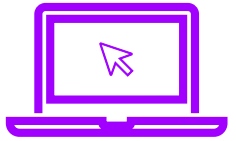
**Step 3** Test Method → Open AccenSportz
URL: **https://lkmdemoaut.accenture.com/AccenSportz/#/**

**Step 4** Post-conditions → Close the Edge browser

Copyright © 2024 Accenture. All rights reserved.

# Web Automation with WebDriver and TestNG (5)

## Demo Solution: Executing Selenium Test Scripts in Edge Browser

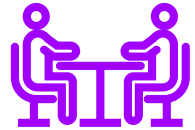


```java
package demo1;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.edge.EdgeDriver;
import org.testng.annotations.AfterClass;
import org.testng.annotations.BeforeClass;
import org.testng.annotations.Test;

Run All
public class Browserlaunch {
    WebDriver driver;

    @BeforeClass
    public void beforeclass() {
        // Initiating browser
        driver = new EdgeDriver();
        // maximize the browser
        driver.manage().window().maximize();
    }

    @Test
    Run | Debug
    public void test01() {
        // Navigating to the application
        driver.get("https://lkmdemoaut.accenture.com/AccenSportz/#/");
    }

    @AfterClass
    public void afterclass() {
        // closing the browser
        driver.close();
    }

}
```

Copyright © 2024 Accenture. All rights reserved.

# Web Automation with WebDriver and TestNG (6)

## Demo: Executing Selenium Test Scripts in Firefox Browser



- Priya has learned the Selenium WebDriver commands to automate the Firefox browser
- She decides to create a new Maven project and design a script to automate the browser
- Scenario:

**Step 1** Create a TestNG class to write the test automation webdriver commands

**Step 2** Pre-requisites → Launch Firefox browser and maximize the browser window

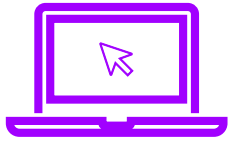
**Step 3** Test Method → Open AccenSportz
URL: https://lkmdemoaut.accenture.com/AccenSportz/#/

**Step 4** Post-conditions → Close the Firefox browser

Copyright © 2024 Accenture. All rights reserved.

# Web Automation with WebDriver and TestNG (7)

## Demo Solution: Executing Selenium Test Scripts in Firefox Browser



```java
package demo1;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.firefox.FirefoxDriver;
import org.testng.annotations.AfterClass;
import org.testng.annotations.BeforeClass;
import org.testng.annotations.Test;

Run All
public class Browserlaunch {
    WebDriver driver;

    @BeforeClass
    public void beforeclass() {
        // Initiating browser
        driver = new FirefoxDriver();
        // maximize the browser
        driver.manage().window().maximize();
    }

    @Test
    Run | Debug
    public void test01() {
        // Navigating to the application
        driver.get("https://lkmdemoaut.accenture.com/AccenSportz/#/");
    }

    @AfterClass
    public void afterclass() {
        // closing the browser
        driver.close();
    }

}
```

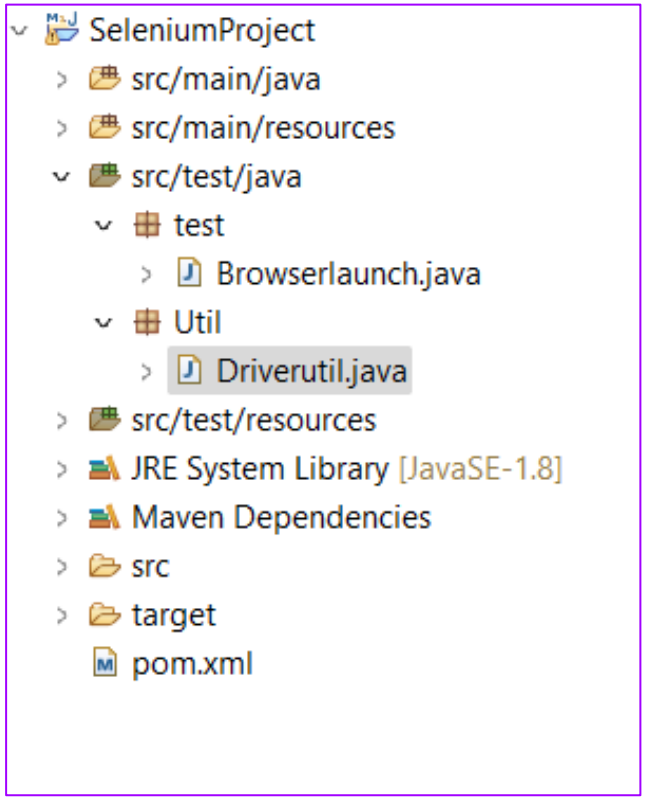
Copyright © 2024 Accenture. All rights reserved.

# Browser Utility Class

# Browser Utility Class (1)

## Introduction to Utility Class

- A Utility class can be created to achieve code reusability
- We can create a browser utility class to achieve reusability for testing in multiple browsers
- This utility class will create driver instances of all the browsers, as per the user input (browser name).
- Follow the project structure as shown in the picture to adhere to the coding standards.




Copyright © 2024 Accenture. All rights reserved.

# Browser Utility Class (2)

## Use of Browser Utility Class

- Design a separate java class and name it as DriverUtil in a separate util package. Create a static method to instantiate the desired browser based on the browser name as an input to this utility function as shown below.

```
1  package Util;
2
3  import org.openqa.selenium.WebDriver;
4  import org.openqa.selenium.chrome.ChromeDriver;
5  import org.openqa.selenium.edge.EdgeDriver;
6  import org.openqa.selenium.firefox.FirefoxDriver;
7
8  public class Driverutil {
9
10     static public WebDriver BrowserInstance(String browsername) {
11
12         if (browsername.equalsIgnoreCase("chrome")) {
13             return new ChromeDriver();
14
15         } else if (browsername.equalsIgnoreCase("edge")) {
16             return new EdgeDriver();
17
18         } else if (browsername.equalsIgnoreCase("firefox")) {
19             return new FirefoxDriver();
20         } else {
21
22             return null;
23
24         }
25
26     }
27
28 }
```

Copyright © 2024 Accenture. All rights reserved.

# Browser Utility Class (3)

## Use of Browser Utility Class

- User can create a driver instance for a choice of browser, by just specifying the browser name and call the necessary Utility Class method.

```java
package test;

import org.openqa.selenium.WebDriver;
import org.testng.annotations.AfterClass;
import org.testng.annotations.BeforeClass;
import org.testng.annotations.Test;

import Util.Driverutil;

Run All
public class Browserlaunch {
    WebDriver driver;

    @BeforeClass
    public void beforeclass() {
        // Initiating browser using Utility class
        driver = Driverutil.BrowserInstance("edge");
        // maximize the browser
        driver.manage().window().maximize();
    }

    @Test
    Run | Debug
    public void test01() {
        // Navigating to the application
        driver.get("https://lkmdemoaut.accenture.com/AccenSportz/#/");
    }

    @AfterClass
    public void afterclass() {
        // closing the browser
        driver.close();
    }

}
```

Copyright © 2024 Accenture. All rights reserved.

# Knowledge Check

## Question 01

Which of the following components identify the artifacts in Maven?

a) Artifact Id

b) Group ID

c) Version

d) All the above

Copyright © 2024 Accenture. All rights reserved.

# Knowledge Check

## Question 02

What is the correct syntax to instantiate a Firefox browser session?

a) WebDriver driver = new FirefoxDriver();

b) WebDriver driver = new Firefox();

c) WebDriver driver = new GeckoDriver();

d) None of the above

Copyright © 2024 Accenture. All rights reserved.

# Module Summary

Now, you will be able to:

- Understand about WebDriver Architecture
- Demonstrate how to use Maven Project with Dependencies
- Demonstrate Web Automation with WebDriver and TestNG
- Demonstrate how to design Browser Utility Class

Copyright © 2024 Accenture. All rights reserved.

# Thank You